

```
(root@wick)~[/home/wick/terraform]
# terraform init

Initializing the backend...

Initializing provider plugins...
- Finding kreuzwerker/docker versions matching "→ 3.0.1" ...
- Installing kreuzwerker/docker v3.0.2...
- Installed kreuzwerker/docker v3.0.2 (self-signed, key ID BD080C4571C6104C)

Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://www.terraform.io/docs/cli/plugins/signing.html

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

as shown in Figure 2.

`terraform init`

After `terraform init`, execute the command `terraform apply` to check the settings and apply; now the instance is ready to begin (Figure 3).

On running the command `docker ps`, which lists the running Docker container, we see the nginx instance running.

Once the instance is up and running, we can use `terraform destroy` to terminate the process and its instance.

We hope this initial encounter with Terraform leads you to explore community-driven modules, real-world case studies, and best practices

Figure 2: Terraform init

```
(root@wick)~[/home/wick/terraform]
# terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# docker_container.nginx will be created
+ resource "docker_container" "nginx" {
  + attach              = false
  + bridge              = (known after apply)
  + command             = (known after apply)
  + container_logs      = (known after apply)
  + container_read_refresh_timeout_milliseconds = 15000
  + entrypoint          = (known after apply)
  + env                = (known after apply)
  + exit_code           = (known after apply)
  + hostname            = (known after apply)
  + id                  = (known after apply)
  + image               = (known after apply)
  + init                = (known after apply)
  + ipc_mode            = (known after apply)
  + log_driver          = (known after apply)
  + logs                = false
  + must_run            = true
  + name                = "tf-nginx"
  + network_data        = (known after apply)
  + read_only           = false
  + remove_volumes     = true
  + restart             = "no"
  + rm                  = false
  + runtime             = (known after apply)
  + security_opts       = (known after apply)
  + shm_size            = (known after apply)
  + start               = true
  + stdin_open          = false
  + stop_signal         = (known after apply)
  + stop_timeout        = (known after apply)
  + tty                 = false
  + wait                = false
  + wait_timeout        = 60

  + ports {
    + external = 8000
    + internal = 80
    + ip       = "0.0.0.0"
    + protocol = "tcp"
  }
}
```

Figure 3: Output of `terraform apply`